

Solution Code - PL SQL programming - Error_Handling

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: solution.sql

```
1 | -- Scenario 1: SafeTransferFunds Stored Procedure
2 | CREATE OR REPLACE PROCEDURE SafeTransferFunds(
3 |     p_from_acc IN NUMBER,
4 |     p_to_acc   IN NUMBER,
5 |     p_amount   IN NUMBER
6 | ) IS
7 |     v_balance NUMBER;
8 |     insufficient_funds EXCEPTION;
9 | BEGIN
10 |     SELECT Balance INTO v_balance FROM Accounts WHERE AccountID = p_from_acc FOR UPDATE;
11 |     IF v_balance < p_amount THEN
12 |         RAISE insufficient_funds;
13 |     END IF;
14 |
15 |     UPDATE Accounts SET Balance = Balance - p_amount WHERE AccountID = p_from_acc;
16 |     UPDATE Accounts SET Balance = Balance + p_amount WHERE AccountID = p_to_acc;
17 |     COMMIT;
18 | EXCEPTION
19 |     WHEN insufficient_funds THEN
20 |         ROLLBACK;
21 |         DBMS_OUTPUT.PUT_LINE('Error: Insufficient funds in account ' || p_from_acc);
22 |     WHEN NO_DATA_FOUND THEN
23 |         ROLLBACK;
24 |         DBMS_OUTPUT.PUT_LINE('Error: Account not found.');
```

```
25 |     WHEN OTHERS THEN
26 |         ROLLBACK;
27 |         DBMS_OUTPUT.PUT_LINE('SQL Error: ' || SQLERRM);
28 | END;
29 | /
30 |
31 | -- Scenario 2: UpdateSalary Stored Procedure
32 | CREATE OR REPLACE PROCEDURE UpdateSalary(
33 |     p_emp_id IN NUMBER,
34 |     p_pct    IN NUMBER
35 | ) IS
36 | BEGIN
37 |     UPDATE Employees SET Salary = Salary * (1 + p_pct/100) WHERE EmployeeID = p_emp_id;
38 |     IF SQL%ROWCOUNT = 0 THEN
39 |         RAISE NO_DATA_FOUND;
40 |     END IF;
41 |     COMMIT;
42 | EXCEPTION
43 |     WHEN NO_DATA_FOUND THEN
```

Solution Code - PL SQL programming - Error_Handling

```
44 |         DBMS_OUTPUT.PUT_LINE('Error: Employee ID ' || p_emp_id || ' does not exist.');
```

```
45 |     WHEN OTHERS THEN
```

```
46 |         DBMS_OUTPUT.PUT_LINE('SQL Error: ' || SQLERRM);
```

```
47 | END;
```

```
48 | /
```

```
49 |
```

```
50 | -- Scenario 3: AddNewCustomer Stored Procedure
```

```
51 | CREATE OR REPLACE PROCEDURE AddNewCustomer(
```

```
52 |     p_id     IN NUMBER,
```

```
53 |     p_name   IN VARCHAR2,
```

```
54 |     p_dob    IN DATE,
```

```
55 |     p_bal    IN NUMBER
```

```
56 | ) IS
```

```
57 |     duplicate_customer EXCEPTION;
```

```
58 |     PRAGMA EXCEPTION_INIT(duplicate_customer, -1); -- Unique constraint violation
```

```
59 | BEGIN
```

```
60 |     INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
```

```
61 |     VALUES (p_id, p_name, p_dob, p_bal, SYSDATE);
```

```
62 |     COMMIT;
```

```
63 | EXCEPTION
```

```
64 |     WHEN DUP_VAL_ON_INDEX OR duplicate_customer THEN
```

```
65 |         DBMS_OUTPUT.PUT_LINE('Error: Customer ID ' || p_id || ' already exists.');
```

```
66 |     WHEN OTHERS THEN
```

```
67 |         DBMS_OUTPUT.PUT_LINE('SQL Error: ' || SQLERRM);
```

```
68 | END;
```

```
69 | /
```